



Software design and testing With C#

SDT621

Table of Contents

TASK 1 – SOFTWARE TESTING STRATEGY DESIGN	2
1.1 Testing Objectives	2
1.2 testing types identification	4
1.3 Testing Environment Setup	4
Task 2 :Test Case Design	5
Task 3:Functional Test Execution.....	6
Task 4: Non Functional Testing.....	10
Task 5 :.....	12

TASK 1 – SOFTWARE TESTING STRATEGY DESIGN

1.1 Testing Objectives

System Purpose

The purpose of the Umoja Learning Management System (ULMS) is to provide a coordinated platform for managing academic processes within the institution , The system aims to improve efficiency, accuracy, and accessibility of academic information.

The system is designed to support:

- Student registration
- Login authentication
- Course enrolment
- Marks capture
- Report generation

Testing Goals

Goals of software testing for ULMS are to:

- Verify that all system features function according to requirements
- Identify and remove defects before deployment
- Confirm user inputs and system responses
- Ensure data accuracy in marks and report calculations
- Confirm secure login authentication
- Measure system performance and reliability
- Improve overall user experience and usability

Scope of Testing

Modules Included

- Login Authentication Module
- Student Registration Module

- Course Enrolment Module
- Marks Capture Module
- Average Calculation Module
- Report Generation Module

Testing Activities Included

- Functional testing
- Non-functional testing
- Input validation testing
- Integration testing
- Security testing
- Performance testing
- Usability testing

Excluded from Scope

- Third-party integrations
- Production cloud deployment
- External payment systems

1.2 testing types identification

Testing Type	Testing Approach	Example
Functional Testing	confirm that all features are working according to system requirements by testing user actions and outputs	Login validation
Input Validation Testing	Ensure that the system accepts only correct data formats and prevents invalid input	Marks entry validation
Integration Testing	You must test communication and interaction between modules to ensure data flows correctly	Course allocation between student and course modules
Performance Testing	You must Measure system speed, response time, and stability under load conditions	Report loading performance
Usability Testing	Evaluate ease of use, navigation, and user friendliness of the application	Navigation between forms
Security Testing	Confirm authentication and protection against unauthorized access	User login authentication

1.3 Testing Environment Setup

Hardware Requirements

We use these types of hardware during testing:

- Intel core i5/i7 processor
- Keyboard and mouse
- Stable internet connection
- Hard drive or SSD

Software Requirements

These are the software environments that were used for testing:

- Windows 11
- Microsoft Visual Studio 2022
- .NET Framework
- Github

Browser

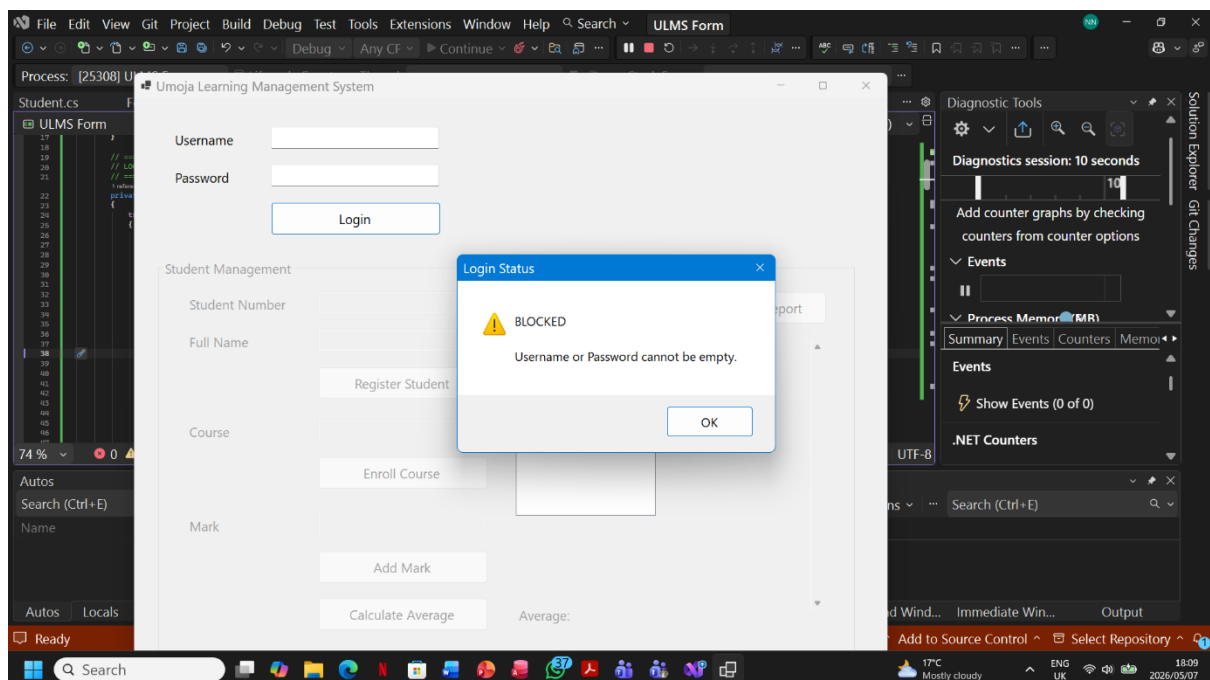
- Operating System- Windows 11
- Browser- Microsoft Edge
- Development IDE – Visual Studio 2022

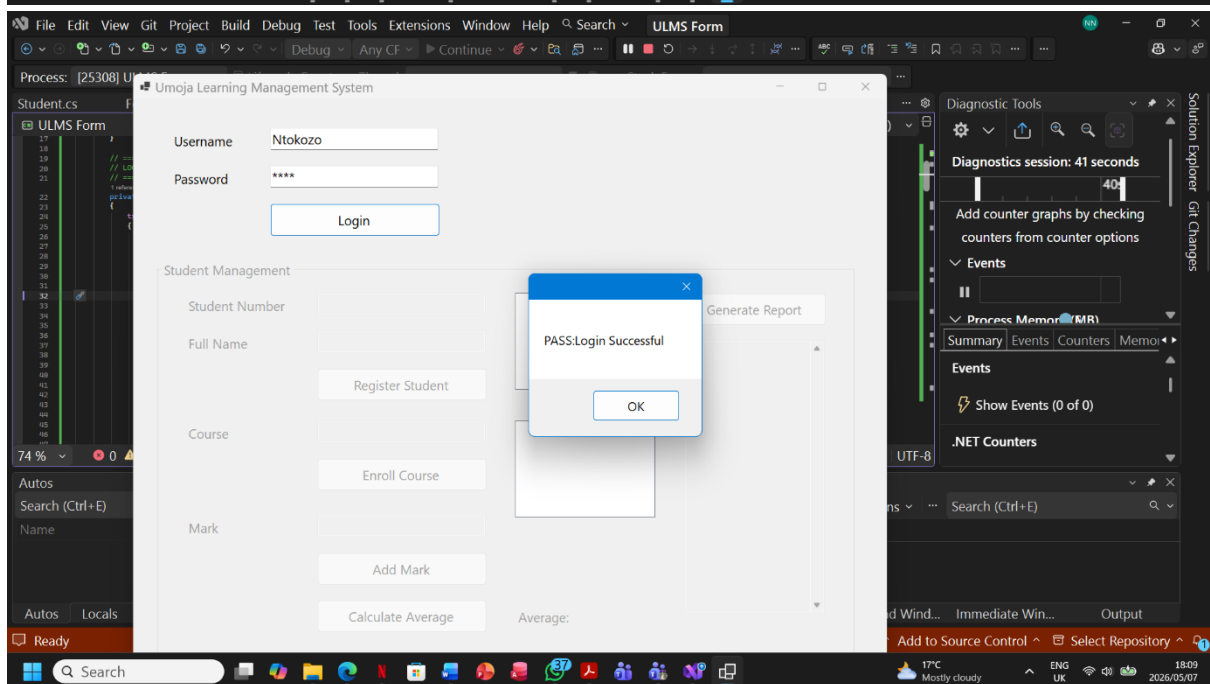
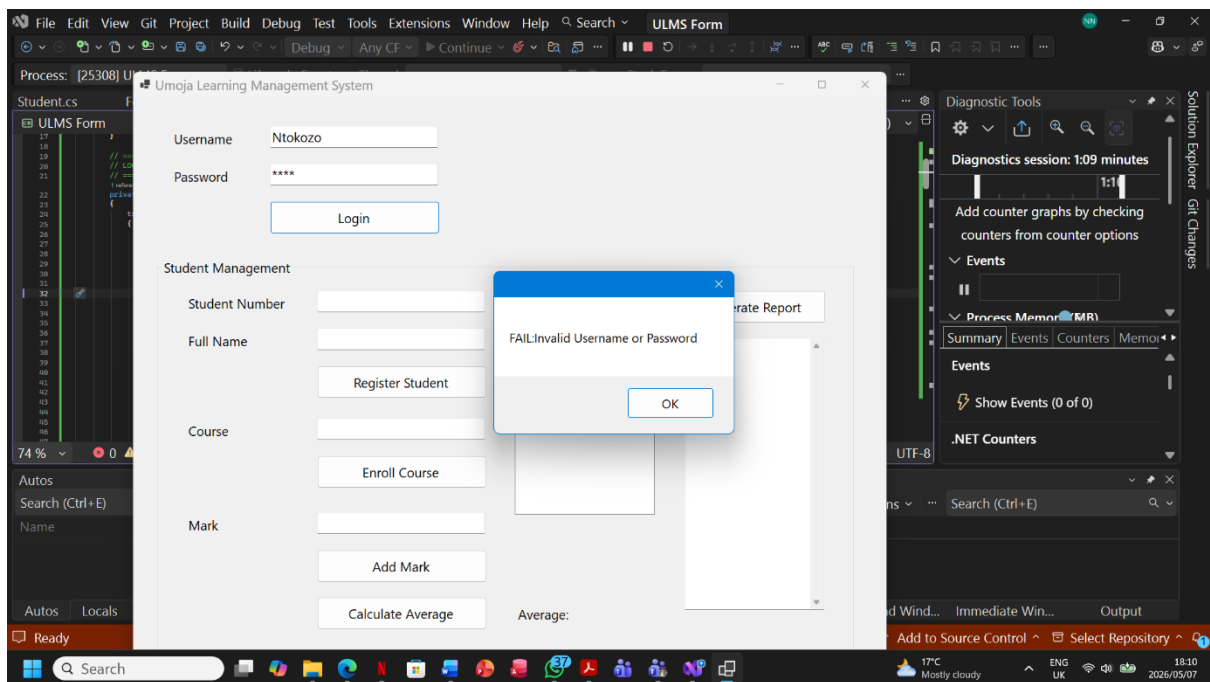
Task 2 :Test Case Design

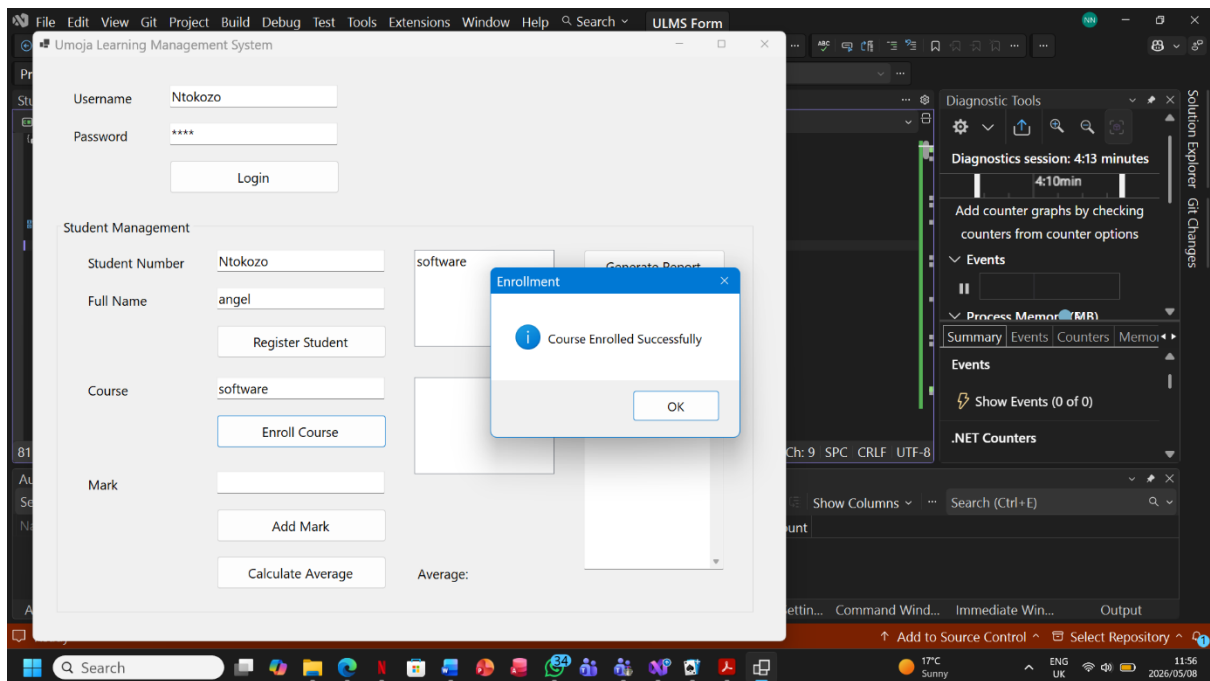
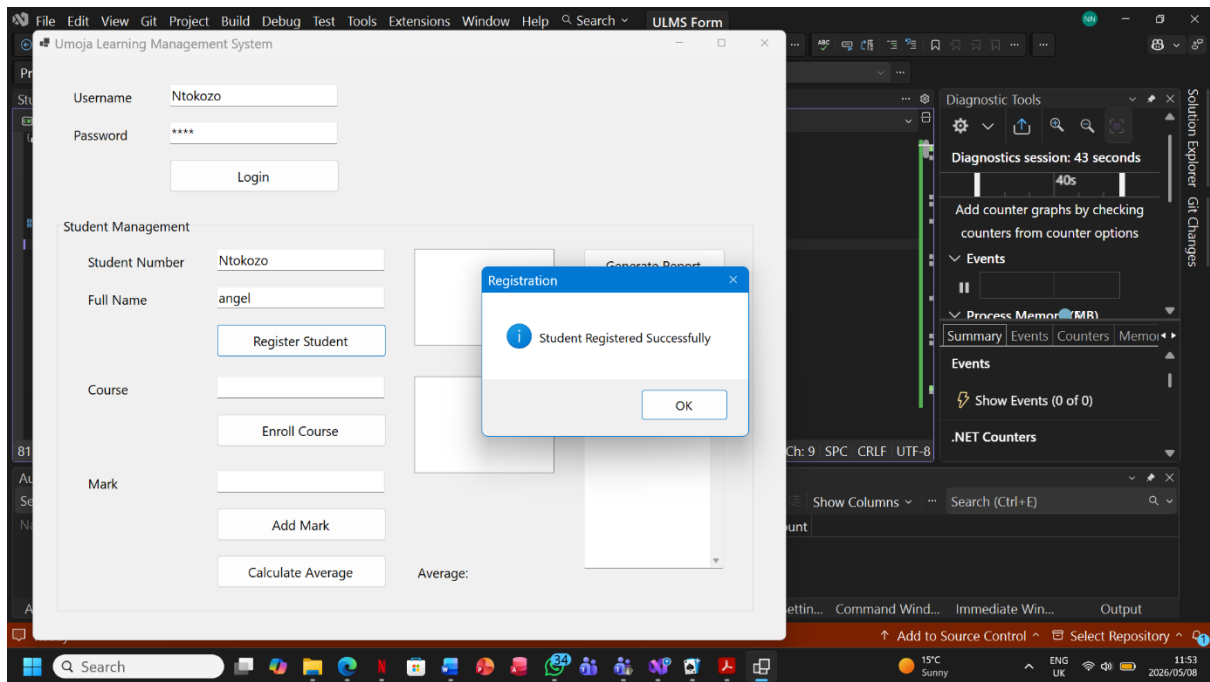
Test Case ID	Description	Input	Expected Output	Actual Output	Status
TC01	Confirm valid student login	Correct username and password	User successfully logs into the system	Login successful	Pass
TC02	Confirm invalid student login	Correct username and incorrect password	System displays "Invalid Login"	Error message displayed	Pass
TC03	Confirm login with empty fields	Empty username and password	Validation message displayed	Validation message displayed	Pass
TC04	Confirm duplicate course enrolment prevention	Student enrolls in same course twice	System prevents duplicate enrolment	Duplicate blocked	Pass
TC05	Confirm successful course enrolment	Student selects a new valid course	Course successfully enrolled	Course added successfully	Pass
TC06	Confirm numeric validation in marks capture	Enter numeric marks (75)	Marks saved successfully	Marks saved	Pass

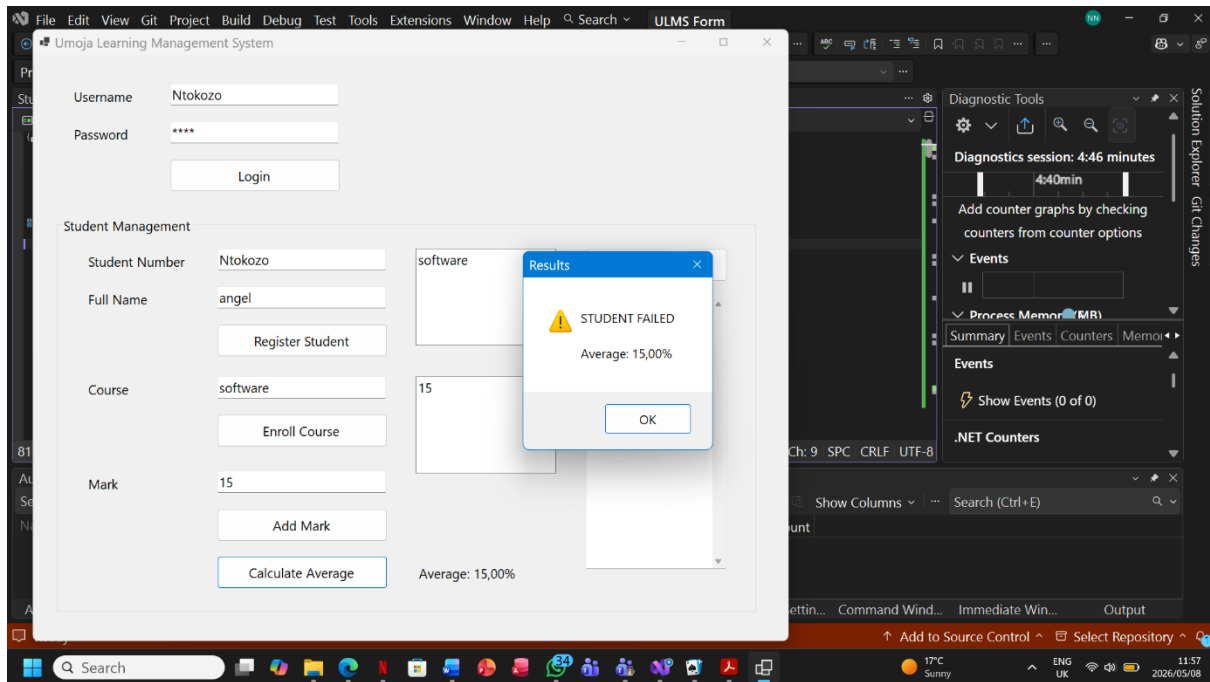
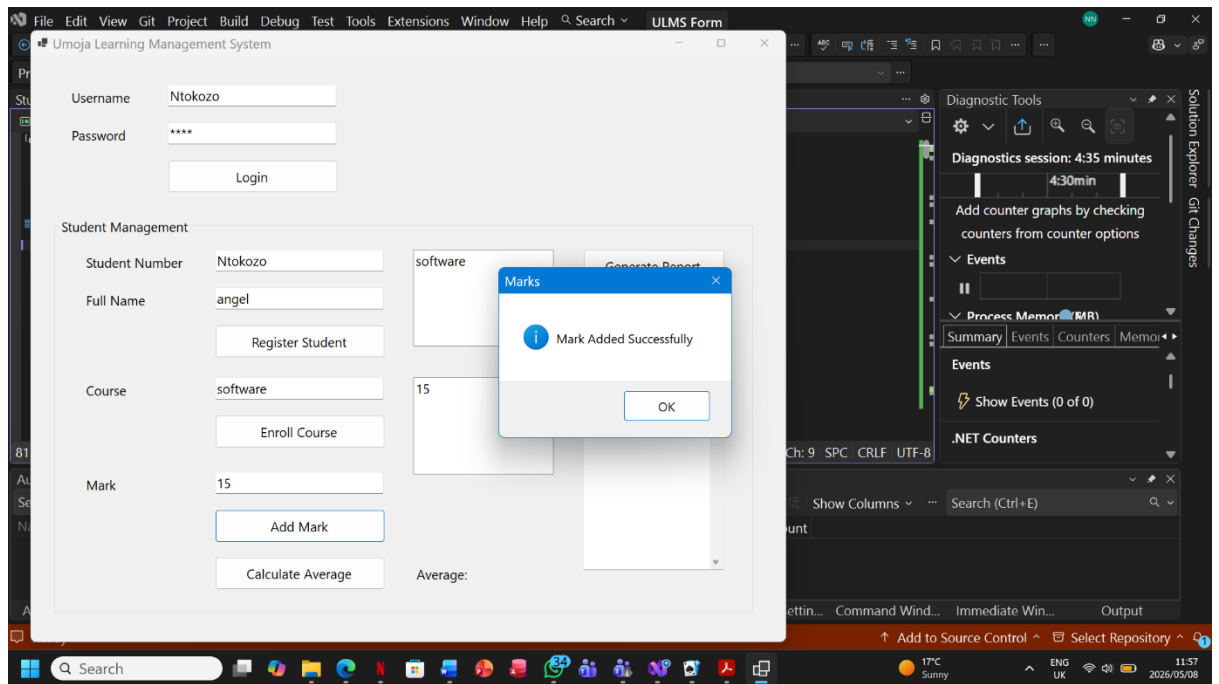
TC07	Confirm non-numeric validation in marks capture	Enter "ABC" as marks	Validation error message displayed	Application crashed	Fail
TC08	Confirm average calculation using boundary values	Enter 0 and 100	Average correctly calculated as 50	Incorrect average displayed	Fail
TC09	Confirm report generation output correctness	Generate student performance report	Correct report generated with correct data	Report generated correctly	Pass
TC10	confirm performance of report generation	Generate large report dataset	Report loads within acceptable time	Report loaded slowly	Fail

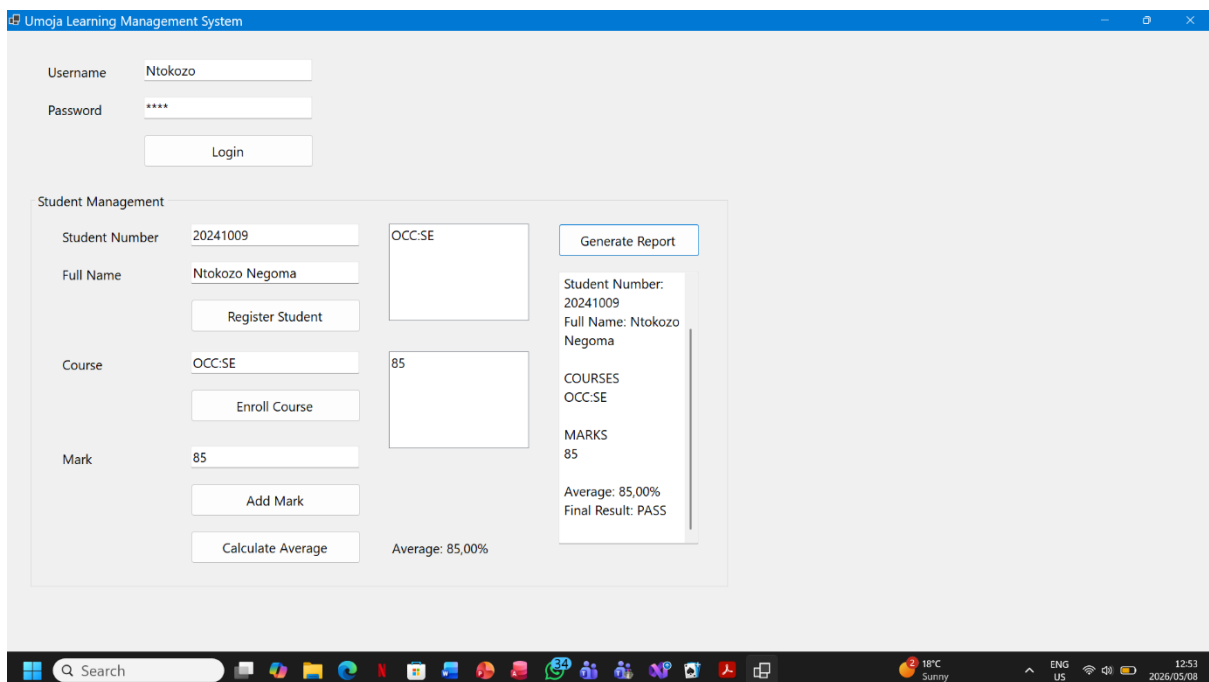
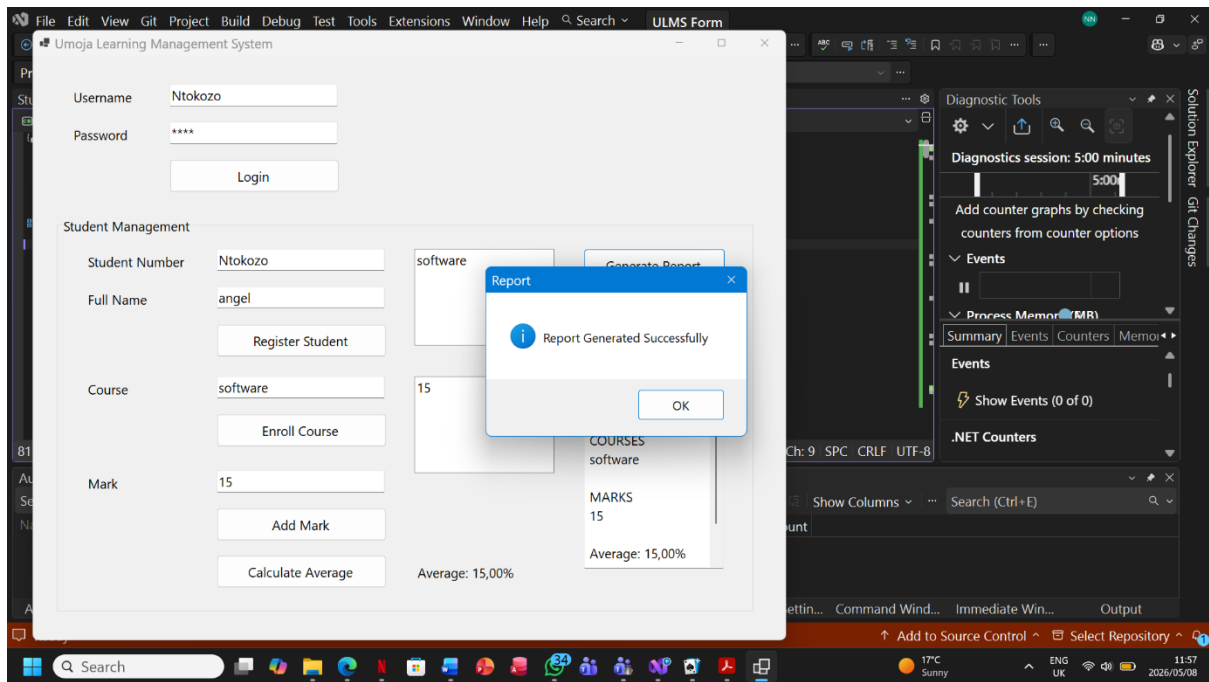
Task 3:Functional Test Execution



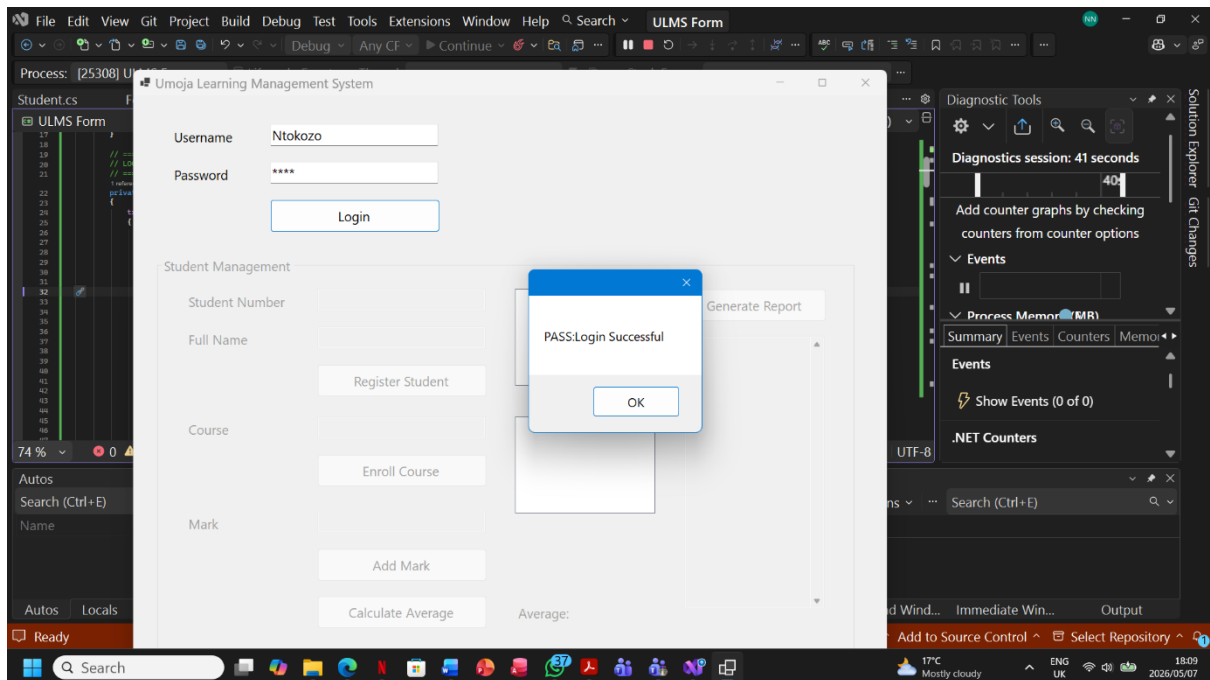
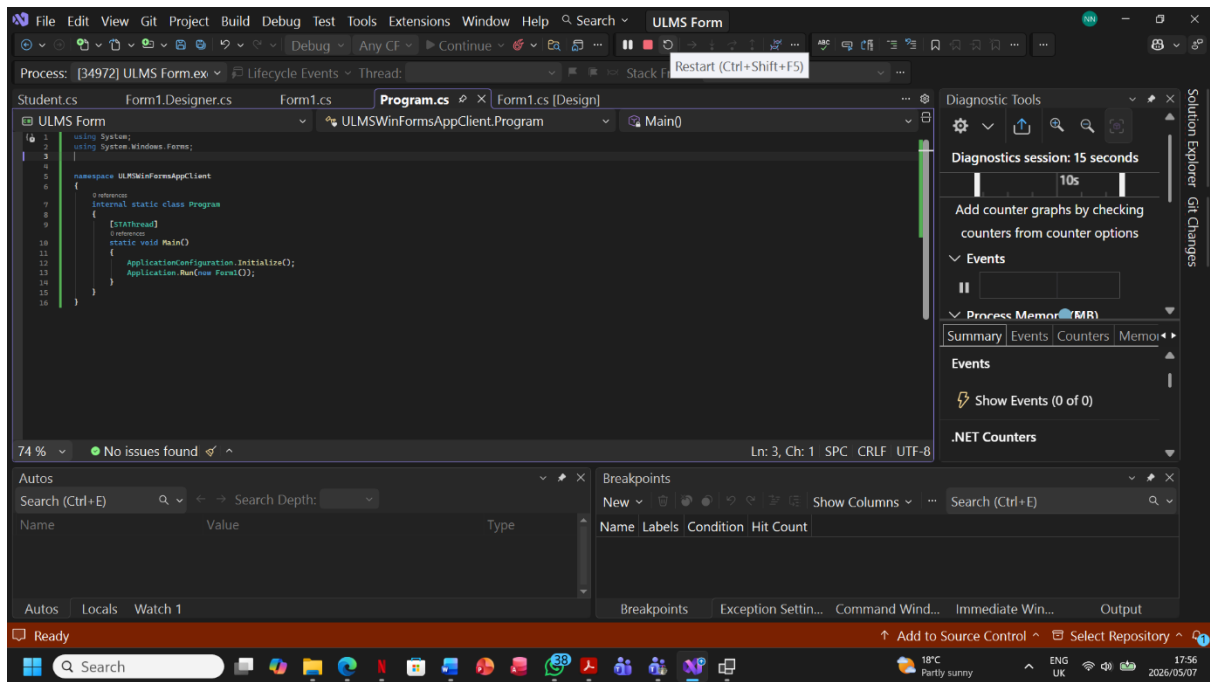




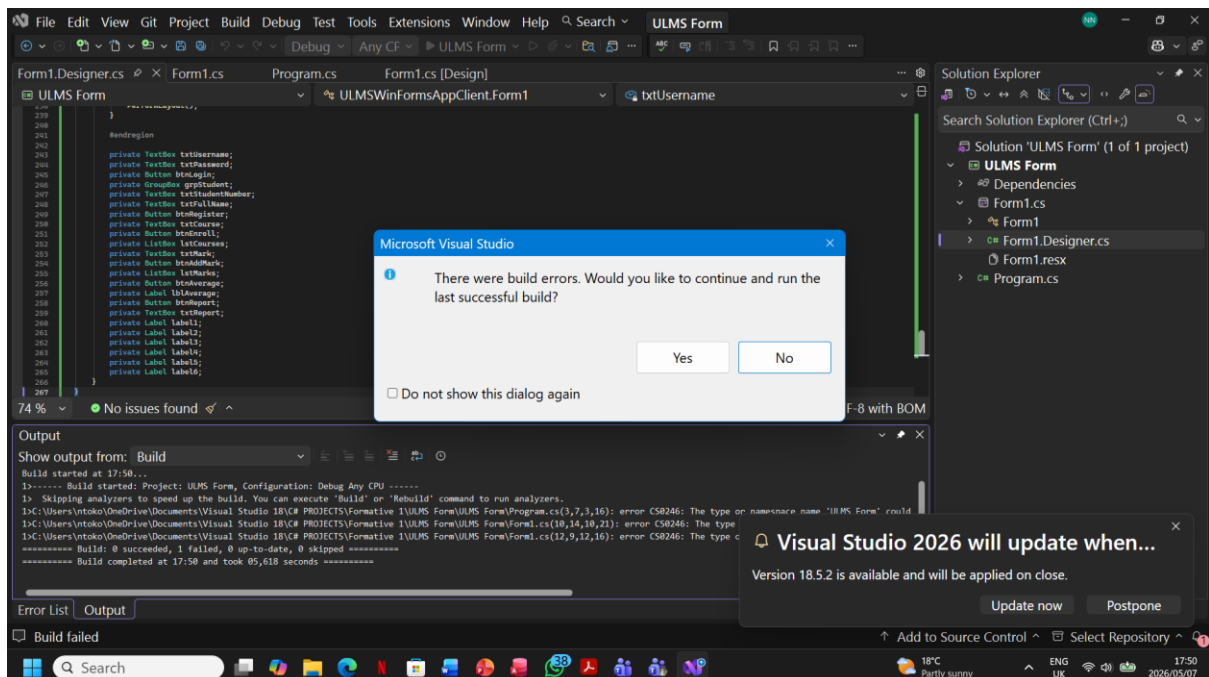
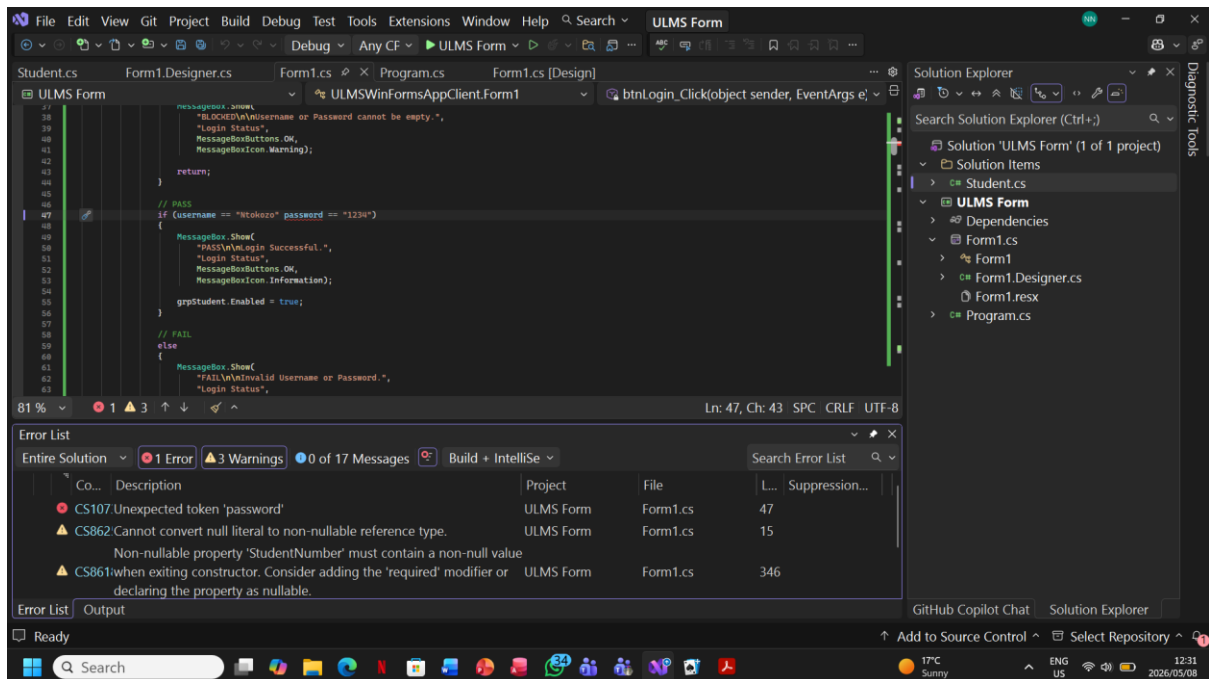


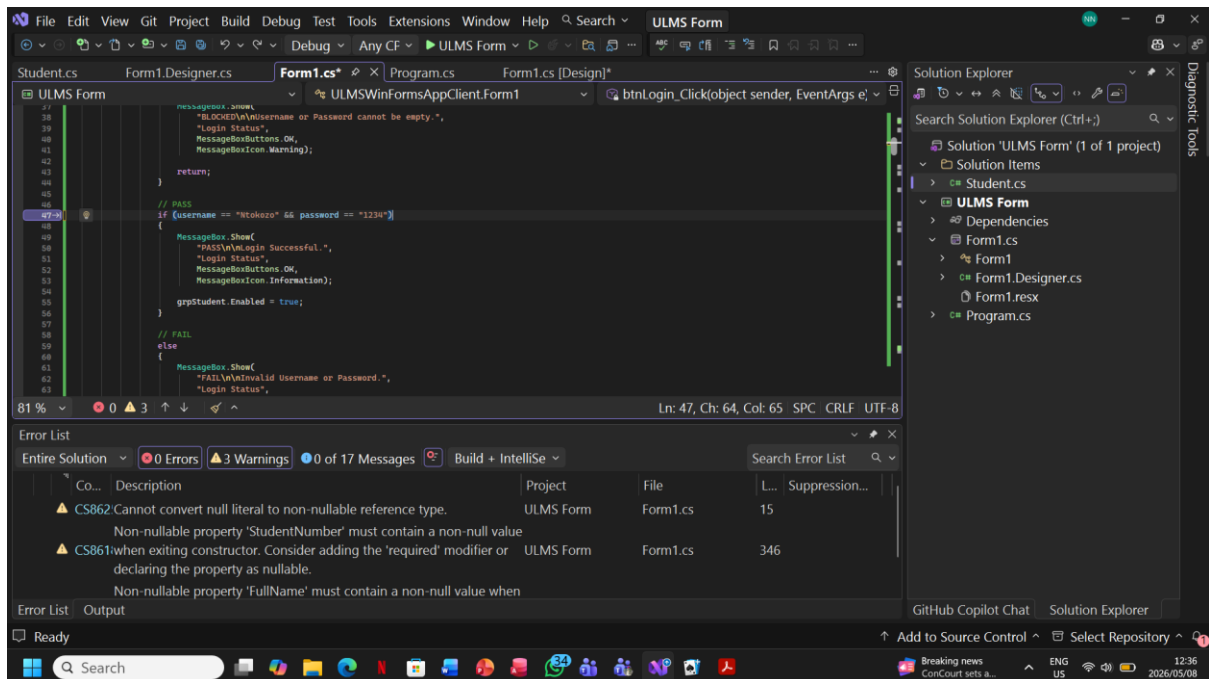


Task 4: Non Functional Testing



Task 5 : DEBUGGING SOURCE CODE





The login validation code originally was having an error because there was no logical operator between the username condition and the password condition So I fixed it when I include the AND (&&) Operator

Task 6: DEFECT REPORTING

Bug ID	Module Affected	Description of Defect	Severity Level	Priority Level	Status	Corrective Action Taken
BUG001	Login Authentication	Missing logical operator between username and password validation caused login failure and compilation errors.	High	High	Fixed	Added AND (&&) operator between conditions.
BUG002	Average Calculation	Incorrect average calculation due to operator precedence	High	High	Fixed	Added brackets to calculate the average correctly.

		produced wrong student averages.				
BUG003	Marks Capture	Application crashed when non-numeric values were entered into the marks field.	High	Medium	Fixed	Added try-catch exception handling and validation messages.
BUG004	Course Enrolment	Students could enroll for the same course multiple times.	Medium	Medium	Fixed	Added duplicate validation using the Contains() method.
BUG005	Report Generation	Large reports loaded slowly and reduced system responsiveness.	Medium	Medium	Open	Recommended report optimization and improved data processing.